Algorithms Project: Distinct Elements

1. Introduction

The objective of this project is to determine the number of unique values within a given zero-indexed array A consisting of N integers. This is a fundamental problem in data processing and algorithmic analysis.

2. Problem Constraints

- Number of elements (N): [0..100,000].
- Element range: [-1,000,000..1,000,000].

3. Algorithm 1: Sorting Approach (Non-Recursive)

3.1 Strategy

This algorithm organizes the array using a bubble sort mechanism. Once sorted, identical elements are adjacent, allowing a single linear pass to count unique values.

3.2 Pseudo code

```
ALGORITHM solution_sorting(A):
    n = length of A
    IF n IS 0 THEN RETURN 0

    // Bubble Sort Process
    FOR i FROM 0 TO n-1 DO:
        FOR j FROM 0 TO n-i-2 DO:
            IF A[j] > A[j+1] THEN:
                temp = A[j]
                A[j] = A[j+1]
                A[j+1] = temp
            END IF
```

```
        END FOR
    END FOR

    // Counting Unique Elements
    distinct_count = 1
    FOR i FROM 1 TO n-1 DO:
        IF A[i] IS NOT EQUAL TO A[i-1] THEN:
            distinct_count = distinct_count + 1
        END IF
    END FOR
    RETURN distinct_count
END ALGORITHM
```

3.3 Complexity Analysis

- **Time Complexity:** $O(N^2)$ due to nested loops in sorting.
- **Space Complexity:** O(1) for in-place sorting.

4. Algorithm 2: Merge Sort Approach (Recursive)

4.1 Strategy

This approach utilizes the Merge Sort algorithm to recursively divide the array into halves until single-element arrays are reached, then merges them in sorted order. Finally, it counts unique elements via a linear scan.

4.2 Pseudo code

```
ALGORITHM MergeSortDistinct(A):
    IF length(A) == 0 THEN RETURN 0

    // Step 1: Sort the array recursively
    SortedA = MergeSort(A)

    // Step 2: Linear scan to count unique elements
    DistinctCount = 1
```

```
    FOR i FROM 1 TO length(SortedA) - 1 DO
        IF SortedA[i] != SortedA[i-1] THEN
            DistinctCount = DistinctCount + 1
        END IF
    END FOR
    RETURN DistinctCount
END ALGORITHM

ALGORITHM MergeSort(arr):
    IF length(arr) <= 1 THEN RETURN arr

    Mid = length(arr) / 2
    LeftHalf = arr[0...Mid-1]
    RightHalf = arr[Mid...End]

    Left = MergeSort(LeftHalf)
    Right = MergeSort(RightHalf)

    RETURN Merge(Left, Right)
END ALGORITHM

ALGORITHM Merge(left, right):
    Result = empty list
    i = 0, j = 0
    WHILE i < length(left) AND j < length(right) DO
        IF left[i] < right[j] THEN
            Append left[i] to Result
            i = i + 1
        ELSE
            Append right[j] to Result
            j = j + 1
        END IF
    END WHILE
    WHILE i < length(left) DO
        Append left[i] to Result
        i = i + 1
    END WHILE
```

```
    WHILE j < length(right) DO
        Append right[j] to Result
        j = j + 1
    END WHILE
    RETURN Result
END ALGORITHM
```

4.3 Complexity Analysis

- **Time Complexity:** O(N log N) for sorting plus O(N) for the scan.
- **Space Complexity:** O(N) for the merging process.

5. Comparison and Recommendation

Feature	Bubble Sort (Non-Recursive)	Merge Sort (Recursive)
Time Complexity	$O(N^2)$	O(N log N)
Space Complexity	O(1)	O(N)
Best Usage	Small datasets where memory is very limited.	Large datasets (up to 100,000 elements).

> **Conclusion:** Given the problem constraints (N up to 100,000), the **Merge Sort** approach is highly recommended due to its efficient time complexity. Bubble Sort would be impractical for the maximum input size.